03 / GUÍA DEL EJERCICIO

EX30 — Rotaciones y matrices 3D

Class 18 · Parte matemática y aplicación física

Objetivo y alcance
Construir desde cero las matrices $\mathbf{R}_x(\theta)$, $\mathbf{R}_y(\theta)$ y $\mathbf{R}_z(\theta)$ usando seno y coseno; aplicarlas a vectores, estudiar por qué importa el orden en 3D y resolver aplicaciones sencillas de fuerzas y cambios de referencia. Un único archivo Python: sin JSON, clases ni `pytest`.

1. Ubicación del ejercicio

```
Python_Engineering/
02_numerical_methods/
algebra/
class_18_matrices/
EX28_matrix_operations.py
EX29_rotation_2D.py
EX30_rotation_3D.py  ← NUEVO
docs/
ES/
EN/
```

Archivo de trabajo: `EX30_rotation_3D.py`. Librería: `import numpy as np`.

2. Hipótesis del modelo y notación

Sistema cartesiano tridimensional **de mano derecha**, con ejes fijos x, y, z; ángulos positivos según la regla de la mano derecha. Los vectores matemáticos se escriben como columnas, pero se implementan mediante arrays NumPy `(3,)`. Las matrices tienen forma `(3,3)`.

Unidades: los apartados A utilizan vectores sin unidades; los apartados B y C trabajan con fuerzas en newtons; el brazo de palanca se expresa en metros y el momento en N m.

3. Concepto que se implementa primero: matrices $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$

Antes de rotar un vector, programar la construcción de **tres matrices distintas**, eligiendo para cada una un ángulo θ:

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix} \qquad c = \cos\theta,\ \ s = \sin\theta$$

$$\mathbf{R}_y(\theta) = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}$$

$$\mathbf{R}_z(\theta) = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

1. Elegir un ángulo en grados y convertirlo a radianes con `np.deg2rad()`.
2. Calcular seno y coseno del ángulo en radianes.
3. Construir la matriz elegida como un array 3×3, respetando signos y posiciones.
4. Cambiar el ángulo o el eje y **reconstruir** la matriz antes de volver a usarla.

No introducir matrices ya calculadas con valores $0, 1, -1$ como solución general. Los valores trigonométricos se obtienen a partir del ángulo. Todavía no es necesario crear una función propia.

4. Parte A0 — Ejemplo algebraico resuelto (verificación)

Datos: $\boldsymbol{a} = (2, 0, 1)^T$, giro activo $+90°$ alrededor de z.

1. Construir $\mathbf{R}_z(90°)$ desde seno y coseno.
2. Multiplicar la matriz por $\boldsymbol{a}$ y comprobar manualmente:

$$\mathbf{R}_z(90°) = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{R}_z(90°)\boldsymbol{a} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}.$$

3. Comprobar que ambas normas valen $\sqrt{5} \simeq 2{,}2361$.
4. Recuperar el vector original mediante $\mathbf{R}_z(90°)^T$ y comparar los arrays con `np.allclose()`.

5. Parte A1 — Rotaciones elementales alrededor de X, Y y Z

Datos: tres casos **independientes**: $\boldsymbol{b} = (1, 2, 0)^T$, $\boldsymbol{c} = (1, 0, 2)^T$ y $\boldsymbol{d} = (0, 3, 1)^T$.

1. Construir $\mathbf{R}_x(+90°)$ y calcular $\boldsymbol{b}' = \mathbf{R}_x\boldsymbol{b}$.
2. Construir $\mathbf{R}_y(+90°)$ y calcular $\boldsymbol{c}' = \mathbf{R}_y\boldsymbol{c}$.
3. Construir $\mathbf{R}_z(-90°)$ y calcular $\boldsymbol{d}' = \mathbf{R}_z\boldsymbol{d}$.
4. Comparar las normas antes y después de cada giro.
5. Para las tres matrices, verificar $\mathbf{R}^T\mathbf{R} \simeq \mathbf{I}_3$ con `np.eye(3)` y `np.allclose()`. Comprobar que $\det \mathbf{R} \simeq 1$.

Resultados de control: $\boldsymbol{b}' = (1, 0, 2)^T$, $\boldsymbol{c}' = (2, 0, -1)^T$, $\boldsymbol{d}' = (3, 0, 1)^T$. Normas respectivas: $\sqrt{5}$, $\sqrt{5}$ y $\sqrt{10}$.

6. Parte A2 — El orden de giro en 3D no es indiferente

Datos: $\boldsymbol{p} = (1, 0, 0)^T$. Dos giros de $+90°$: uno sobre x y otro sobre z, siempre respecto a **ejes globales fijos**.

1. Aplicar primero el giro sobre x y después sobre z:

$$\boldsymbol{p}_{xz} = \mathbf{R}_z(90°)\mathbf{R}_x(90°)\boldsymbol{p}.$$

2. Aplicar primero el giro sobre z y después sobre x:

$$\boldsymbol{p}_{zx} = \mathbf{R}_x(90°)\mathbf{R}_z(90°)\boldsymbol{p}.$$

3. Comparar ambos resultados mediante `np.allclose()`. ¿Son iguales? Explicar por qué importa el orden.
4. Comprobar la norma de ambos vectores y la del original.

Resultados de control: $\boldsymbol{p}_{xz} = (0, 1, 0)^T$ y $\boldsymbol{p}_{zx} = (0, 0, 1)^T$; todas las normas valen 1.

7. Parte B — Ingeniería: fuerza y momento en 3D

Una fuerza de un actuador está inicialmente alineada con el eje global x: $\boldsymbol{F}_0 = (10, 0, 0)^T$ N. Su dirección gira primero $+90°$ respecto al eje global z y después $+90°$ respecto al eje global x.

1. Construir $\mathbf{R}_z(90°)$ y $\mathbf{R}_x(90°)$ a partir de seno y coseno.
2. Calcular las componentes tras el primer giro y después la fuerza final con la multiplicación ordenada $\boldsymbol{F}_G = \mathbf{R}_x\mathbf{R}_z\boldsymbol{F}_0$.
3. Comprobar la conservación de los 10 N de magnitud.
4. La fuerza actúa con brazo $\boldsymbol{r} = (0{,}2, 0, 0)^T$ m. Calcular el momento $\boldsymbol{M} = \boldsymbol{r} \times \boldsymbol{F}_G$ mediante `np.cross()` (EX27).
5. Interpretar los signos y escribir las unidades de fuerza y momento en los resultados.

Resultados de control: $\boldsymbol{F}_1 = (0, 10, 0)^T$ N; $\boldsymbol{F}_G = (0, 0, 10)^T$ N; $\boldsymbol{M} = (0, -2, 0)^T$ N m.

8. Parte C — Cambio de referencia (sin girar la fuerza)

Una fuerza **fija** tiene componentes globales $\boldsymbol{F}_G = (3, 4, 0)^T$ N. Los **ejes locales** giran $+90°$ alrededor de z con respecto a los ejes globales. La fuerza física **no se mueve**.

1. Construir $\mathbf{R}_z(90°)$.
2. Obtener las coordenadas locales con $\boldsymbol{F}_L = \mathbf{R}_z(90°)^T \boldsymbol{F}_G$.
3. Recuperar las coordenadas globales mediante $\mathbf{R}_z(90°)\boldsymbol{F}_L$.
4. Comparar vectores recuperados y normas, e interpretar en una frase qué ha cambiado y qué no.

Resultados de control: $\boldsymbol{F}_L = (4, -3, 0)^T$ N; norma de ambas representaciones: 5 N.

9. Lista de verificación

- Los giros siguen la regla de la mano derecha y se construyen con grados convertidos a radianes.
- $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$ son arrays `(3,3)`; los vectores, arrays `(3,)`.
- Para 3D, se observa que en general $\mathbf{R}_x\mathbf{R}_z \neq \mathbf{R}_z\mathbf{R}_x$.
- Los controles incluyen norma, $\mathbf{R}^T\mathbf{R} \simeq I$, determinante, resultados manuales y recuperación del vector.
- Se diferencian rotación activa (B) y cambio de referencia pasivo (C).

10. Ayudas de código: consultar al final

Herramientas disponibles (sin solución completa): `np.deg2rad`, `np.sin`, `np.cos`, `np.array`, `@`, `.T`, `np.allclose`, `np.eye`, `np.linalg.norm`, `np.linalg.det` y `np.cross`.

```
angle_deg = 90.0
angle_rad = np.deg2rad(angle_deg)
c = np.cos(angle_rad)
s = np.sin(angle_rad)
# Construir Rx, Ry o Rz con c y s segun el eje.
# Comprobar R.T @ R comparandola con np.eye(3).
```